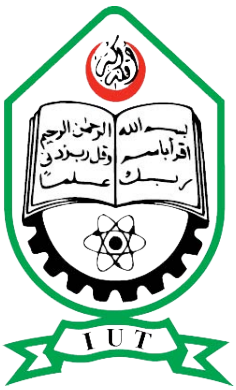
Islamic University of Technology



Digital Image Processing Lab

CSE 4734

Lab 1

Introduction to DIP

Contents

1 Introduction **2**
1.1 Images as Arrays . . . 2

2 Setup **2**

3 Function Reference **3**
3.1 Reading, Converting and Showing Images . . . 3
3.2 NumPy . . . 3
3.3 Matplotlib . . . 4

4 Tasks **5**
4.1 Task 1: Histogram of a Grayscale Image . . . 5
4.2 Task 2: Channel Separation . . . 5
4.3 Task 3: Histograms of a Color Image . . . 6
4.4 Task 4: Color Enhancement . . . 6
4.5 Task 5: Negative of a Grayscale Image . . . 7
4.6 Task 6: Negative of a Color Image . . . 8

1 Introduction

In this lab you will implement basic Digital Image Processing (DIP) operations in Python: computing histograms, separating and enhancing RGB channels, and generating image negatives. No prior experience with OpenCV, NumPy or Matplotlib is assumed. Sections 1.1–3 explain the ideas and the functions you need; Section 4 lists the tasks, each with a sample input and expected output.

1.1 Images as Arrays

A digital image is a grid of **pixels**. In Python it is a **NumPy array**:

- **Grayscale**: shape `(height, width)`, one intensity per pixel.
- **Color**: shape `(height, width, 3)`, one value per color channel.
- Values are of type `uint8`: whole numbers from **0** (black / no color) to **255** (white / full color). There are $L = 256$ levels and $L_{\max} = 255$.
- A pixel is accessed as `img[row, col]`; $(0, 0)$ is the top-left corner. `img[:, :, k]` is the whole channel `k`.

The **histogram** of a grayscale image counts how many pixels have each intensity: $h(k)$ = number of pixels with value k, for $k = 0, \ldots, 255$. Its counts add up to the total number of pixels. Mass on the left means a dark image, on the right a bright one. A color image has one histogram per channel.

The **negative** reverses intensities: $g(x, y) = L_{\max} - f(x, y)$. For color images it is applied to each channel, turning red into cyan, green into magenta and blue into yellow.

Watch out

OpenCV uses BGR order. After `cv2.imread`, channel 0 is **Blue**, 1 is Green, 2 is **Red**. PIL and Matplotlib use RGB (0 = Red). Task 4 numbers channels as RGB. Mixing the two orders is a common bug in this lab.

2 Setup

The notebook runs both in the lab's Jupyter container and in Google Colab without changes. Run this cell first:

```
import cv2
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
from IPython.display import display
from IPython.display import Image as IPyImage

def cv2_imshow(img):
# Show an OpenCV image (grayscale or BGR) inside the notebook.
# Works in Jupyter and Colab.
    success, encoded = cv2.imencode(".png", img)
    if not success:
        raise ValueError("Could not encode image")
display(IPyImage(data=encoded.tobytes()))
```

IPython's `Image` is imported as `IPyImage` so it does not replace PIL's `Image`.

Loading your images. Put the image files in the same folder as the notebook and open them by file name, e.g. `cv2.imread("flower.jpg")`.

- **Lab container**: copy the files into the notebook's folder using the Jupyter file browser.
- **Colab**: drag the files into the *Files* panel (folder icon, left sidebar). They are deleted when the session ends.

3 Function Reference

3.1 Reading, Converting and Showing Images

Function	What it does
`cv2.imread(path)`	Loads a color image as a BGR array `(H, W, 3)`. **Returns `None` if the path is wrong**, without an error.
`cv2.imread(path, cv2.IMREAD_GRAYSCALE)`	Loads as grayscale `(H, W)`.
`np.array(Image.open(path))`	Loads with PIL as an **RGB** array. Add `.convert("L")` before `np.array` for grayscale.
`cv2.cvtColor(img, code)`	Converts color order. Codes: `cv2.COLOR_BGR2RGB`, `cv2.COLOR_RGB2BGR`, `cv2.COLOR_BGR2GRAY`.
`cv2.split(img)`	Splits a 3-channel image into three 2D arrays (in the image's channel order).
`cv2.merge([c0, c1, c2])`	Stacks three equally sized 2D arrays into one 3-channel image.
`cv2_imshow(img)`	Shows a grayscale or **BGR** image in the notebook.
`cv2.imwrite(path, img)`	Saves an image (expects BGR).

```
color = cv2.imread("flower.jpg")
gray = cv2.imread("flower.jpg", cv2.IMREAD_GRAYSCALE)
print(color.shape, gray.shape, color.dtype)   # (480, 640, 3) (480, 640) uint8

rgb = cv2.cvtColor(color, cv2.COLOR_BGR2RGB)  # now channel 0 is red
b, g, r = cv2.split(color)                    # three (480, 640) arrays
```

3.2 NumPy

Function / syntax	What it does
`img.shape`, `img.ndim`, `img.dtype`, `img.size`	Dimensions; number of dimensions (2 = grayscale, 3 = color); data type; total number of values.
`np.zeros(n, dtype=int)`	Array of `n` zeros, e.g. 256 counters for a histogram.
`np.zeros_like(arr)`	Zeros with the same shape and type as `arr`, e.g. an empty channel.
`arr.copy()`	Independent copy. `b = a` does *not* copy; changing `b` changes `a`.
`arr.flatten()`	2D image → 1D array of all pixel values, so one `for` loop visits every pixel.
`counts[k] += 1`	Increments counter `k`. A pixel value (0–255) can be used directly as `k`.
`img[:, :, k]`	Selects (or assigns to) the whole channel `k`.
`255 - img`, `img * 0.5`	Arithmetic applies to *every* element at once; no loop needed.
`arr.astype(np.float32)`	Converts type, e.g. before arithmetic that may leave 0–255.
`np.clip(arr, 0, 255)`	Limits all values to the range 0–255.
`np.sum`, `np.mean`, `np.argmax`	Total, average, and index of the largest value.
`np.arange(256)`	The numbers 0..255, useful as the x-axis of a histogram.

```
small = np.array([[10, 20],
[20, 30]], dtype=np.uint8)
print(small.flatten())    # [10 20 20 30]

for value in small.flatten():
print(value)          # 10, 20, 20, 30
```

Watch out

`uint8` **wraps around instead of stopping at 0 or 255:** `250 + 10` gives `4`, and `5 - 10` gives `251`. Bright pixels suddenly turn dark. When a result can leave 0–255: convert with `.astype(np.float32)`, compute, `np.clip(..., 0, 255)`, then `.astype(np.uint8)`.

3.3 Matplotlib

Function	What it does
`plt.figure(figsize=(w, h))`	Starts a new figure (size in inches).
`plt.plot(y, color="red", label="Red")`	Line graph. Several `plot` calls before `show` draw on the same axes.
`plt.bar(x, heights, width=1)`	Bar graph, a classic histogram look.
`plt.title`, `plt.xlabel`, `plt.ylabel`	Title and axis labels.
`plt.legend()`, `plt.grid(True)`	Shows `label` names; adds a grid.
`plt.subplot(rows, cols, i)`	Selects cell `i` (starting at 1) of a grid of plots.
`plt.imshow(img)`	Draws an image in a plot. Color images must be **RGB**.
`plt.imshow(g, cmap="gray", vmin=0, vmax=255)`	Draws a grayscale image without false colors or contrast stretching.
`plt.axis("off")`, `plt.tight_layout()`	Hides ticks; prevents overlapping titles.
`plt.show()`	Draws the figure.

```
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(color, cv2.COLOR_BGR2RGB))
plt.title("Original")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(gray, cmap="gray", vmin=0, vmax=255)
plt.title("Grayscale")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Watch out

Showing a BGR image with `plt.imshow` swaps red and blue. A 2D array shown without `cmap="gray"` appears purple/yellow. That is Matplotlib's default color map, not a "colored channel".

4 Tasks

For each task, test your function on the sample image and compare your result with the expected output, then try one image of your own.

4.1 Task 1: Histogram of a Grayscale Image

Write a function that takes a grayscale image and returns its histogram (an array of 256 counts), then plot it. Do **not** use `cv2.calcHist`, `np.histogram` or `plt.hist` inside your function.

```
def my_histogram(gray_image):
    # Initialize an array of intensities with all zeros
    # Loop over all pixel values
    # For each pixel, get its intensity
    # Increment the intensity index of the array
    # Return the array
    pass
```

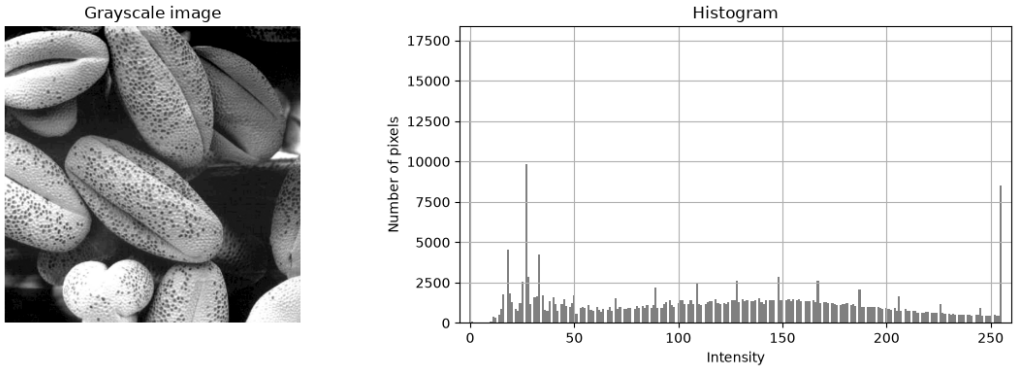


Figure 1: Task 1: grayscale image and its histogram

Hints

How many intensity levels does an 8-bit image have? That sets the array size. **Check:** the counts must add up to `gray_image.size`, and should match `cv2.calcHist([gray_image], [0], None, [256], [0, 256]).flatten()`.

4.2 Task 2: Channel Separation

Divide a color image into its 3 channels and display each **in its own color**: the red channel as a red image whose red intensity at each pixel equals the original pixel's red value, and likewise for green and blue.

```
def channel_show(color_image):
    # implement this function
    pass
```

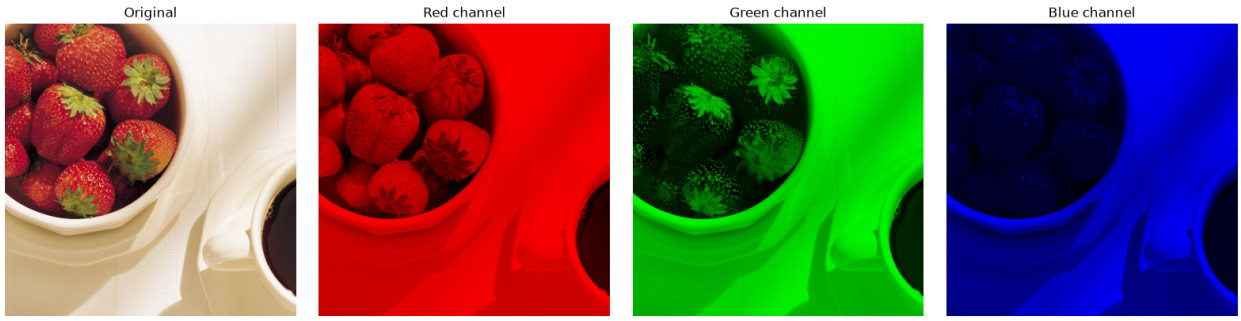


Figure 2: Task 2: red, green and blue channels shown in their own colors

Hints

A single channel on its own is 2D and displays as gray. A "red-only" image is a 3-channel image whose other two channels are zero. Make sure the red values sit in the position your display function treats as red (BGR vs RGB).

4.3 Task 3: Histograms of a Color Image

Use your Task 1 function to compute the three channel histograms of a color image. Plot them with matching colors and a legend (or in three subplots), then state which color is **dominant** and why.

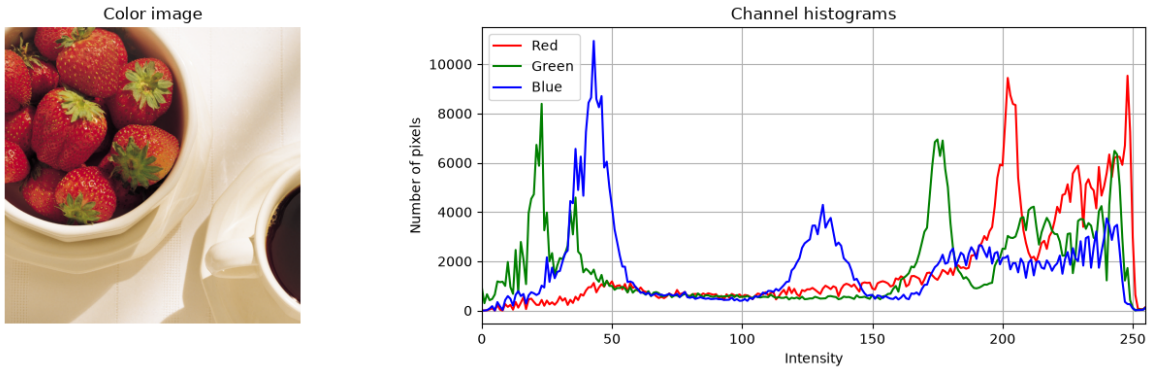


Figure 3: Task 3: color image and its three channel histograms

Hints

Each channel is a 2D array, which is exactly what your Task 1 function accepts. Label curves according to your channel order. Support your conclusion with evidence: where each histogram's mass lies, and e.g. the mean of each channel.

4.4 Task 4: Color Enhancement

Write a function that takes a color image, a `channel_number` (**0 = red, 1 = green, 2 = blue**) and an `enhancement_val` between 0 and 1, and returns an image in which that color is strengthened using the **power-law (gamma) transformation**. Only the chosen channel is transformed:

$$r = \frac{f(x,y)}{L_{\max}}, \qquad s = r^{\gamma}, \qquad g(x,y) = L_{\max}\, s, \qquad \gamma = 1 - \text{enhancement_val}$$

Dividing by $L_{\max} = 255$ maps the channel to $[0, 1]$, the power is applied, and the result is scaled back to $[0, 255]$. Since $0 \le r \le 1$, a $\gamma < 1$ raises the values (dark parts the most), strengthening that color; $\gamma = 1$ leaves the image

unchanged. So an enhancement value of 0 changes nothing and larger values give a stronger effect. Show the original and enhanced images side by side.

```
def color_enhance(color_image, channel_number, enhancement_val):
    # implement this function
    pass
```



Figure 4: Task 4: red enhancement

Hints

Change only the chosen channel and work on a copy. Do the calculation in floating point (`.astype(np.float32)`); `uint8` division and powers give wrong results. The `**` operator (or `np.power`) raises every element to a power at once. Convert the result back to `uint8` before displaying. `channel_number` is in RGB order; if your image is BGR, which index holds red? Try 0.2, 0.5 and 0.8.

4.5 Task 5: Negative of a Grayscale Image

Write a function that returns the negative of a grayscale image:

$$g(x, y) = L_{\max} - f(x, y), \quad L_{\max} = 255$$

```
def invert_gray(gray_image):
    # implement this function
    pass
```

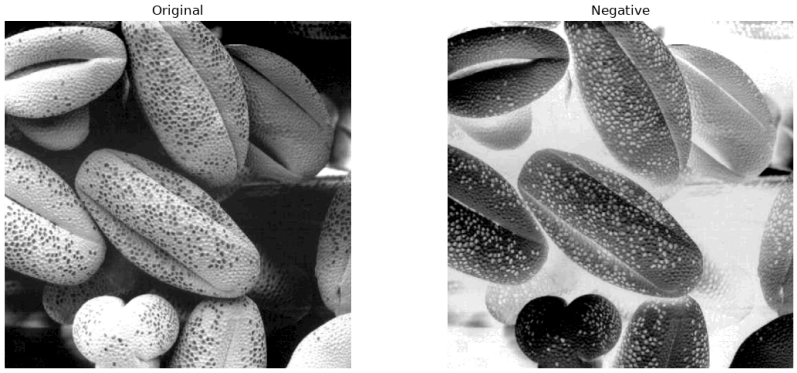


Figure 5: Task 5: grayscale image and its negative

Hints

This can be done with one line of NumPy arithmetic. **Check:** $0 \to 255$, $100 \to 155$, and inverting twice gives back the original. The result must still be `uint8`.

4.6 Task 6: Negative of a Color Image

Extend your function from Task 5 (subsection 4.5) so that it works for color images too.

```
def invert_color(color_image):
    # implement this function
    pass
```

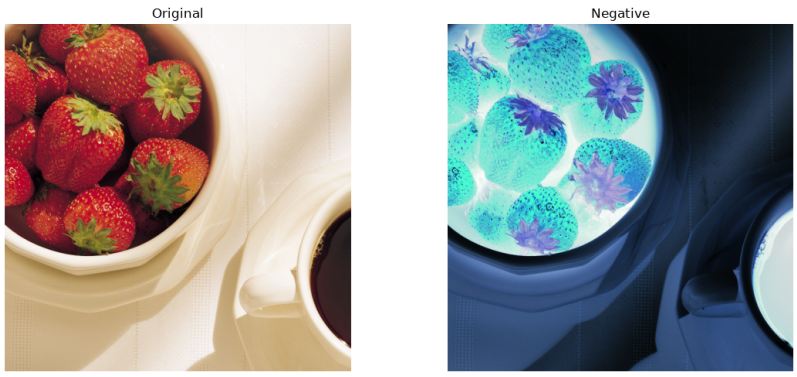


Figure 6: Task 6: color image and its negative

Hints

The formula is applied to every channel independently. **Check:** red becomes cyan, white becomes black.